IBM-Style Cloud Full Stack Mock Assessment

60 questions • 60 minutes • one question at a time • priority: Java + Spring ecosystem + Microservices

TIME REMAINING
42:22

Assessment complete — here are your results

Score	Percentage	Correct
53 / 60	88%	53

Wrong	Unanswered	Avg. time answered
7	0	00:18

Topic-wise performance

Topic	Correct	Total	Accuracy
Java	7	8	88%
HTML/CSS	5	6	83%
JavaScript	6	7	86%
Angular	6	8	75%
React	7	7	100%
MongoDB	6	7	86%
Spring Core/AOP/Testing	4	5	80%
Spring DAO/Hibernate	3	3	100%
Spring Boot/REST	4	4	100%
Microservices	3	3	100%
DevOps	2	2	100%

Question review

Q1 • Java — Wrong

Consider this code: `List<String> xs = new ArrayList<>(); xs.add(null); xs.add("A");` Which statement is correct?

Your answer: D. The code does not compile because generics forbid null
Correct answer: B. The list contains two elements and permits null
Why: ArrayList permits null elements, so both insertions succeed and the size is 2.

Q2 • Java — Correct

What is printed? `Integer a = 128, b = 128; System.out.print(a == b);`

Your answer: B. false
Correct answer: B. false
Why: Integer caching is guaranteed for -128 through 127. 128 values are normally distinct objects, so `==` compares different references.

Q3 • Java — Correct

A method declares `void m() throws IOException`. Which override is legal in a subclass?

Your answer: B. `void m() throws FileNotFoundException`
Correct answer: B. `void m() throws FileNotFoundException`
Why: An overriding method may declare the same checked exception, a narrower checked exception, or no checked exception. FileNotFoundException is narrower than IOException.

Q4 • Java — Correct

Which choice best explains why `String` is immutable in Java?

Your answer: B. Its internal state cannot be changed after construction; operations create new String objects
Correct answer: B. Its internal state cannot be changed after construction; operations create new String objects
Why: String is designed as an immutable value type: its state is not modified after creation, which supports pooling, security, and thread safety.

Q5 • Java — Correct

What is the main advantage of `try-with-resources`?

Your answer: B. It guarantees resources implementing AutoCloseable are closed even when an exception occurs
Correct answer: B. It guarantees resources implementing AutoCloseable are closed even when an exception occurs
Why: try-with-resources automatically closes declared AutoCloseable resources, including when control leaves the block because of an exception.

Q6 • Java — Correct

Which statement about a Java Stream is correct?

Your answer: B. Intermediate operations are generally lazy and execute when a terminal operation runs
Correct answer: B. Intermediate operations are generally lazy and execute when a terminal operation runs
Why: Operations such as filter and map are lazy; the pipeline is evaluated when a terminal operation such as collect or count executes.

Q7 • Java — Correct

Two threads increment a shared `int counter` one million times each using `counter++` without synchronization. What is the most likely result?

Your answer: B. A value less than 2,000,000 due to lost updates
Correct answer: B. A value less than 2,000,000 due to lost updates
Why: counter++ is a read-modify-write sequence and is not atomic. Concurrent updates can overwrite one another, producing lost increments.

Q8 • Java — Correct

Which memory area is primarily used for objects created with `new`?

Your answer: A. Heap
Correct answer: A. Heap
Why: Objects are allocated in the heap; references and local variables may live in stack frames depending on execution and optimization.

Q9 • HTML/CSS — Correct

Which selector has higher CSS specificity?

Your answer: C. `#app .card`
Correct answer: C. `#app .card`
Why: An ID selector contributes more specificity than class or type selectors, so #app .card is higher.

Q10 • HTML/CSS — Correct

Which HTML attribute is most appropriate for an input's accessible name when there is a visible text label?

Your answer: B. A `<label for=...>` associated with the input's id
Correct answer: B. A `<label for=...>` associated with the input's id
Why: A programmatically associated label provides the accessible name and is the standard approach for form controls.

Q11 • HTML/CSS — Wrong

For a flex container, what does `align-items: center` do by default?

Your answer: A. Centers items along the main axis
Correct answer: B. Centers items along the cross axis

Why: align-items controls alignment across the cross axis. In the default row direction, that is the vertical axis.

Q12 • HTML/CSS — Correct

Which CSS property is best for creating a two-dimensional responsive layout of rows and columns?

Your answer: B. display: grid
Correct answer: B. display: grid
Why: CSS Grid is designed for two-dimensional row/column layout.

Q13 • HTML/CSS — Correct

What is true about `position: absolute`?

Your answer: B. The element is removed from normal flow and positioned relative to its containing block
Correct answer: B. The element is removed from normal flow and positioned relative to its containing block
Why: Absolutely positioned elements are removed from normal flow and positioned against a containing block, commonly an ancestor with a non-static position.

Q14 • HTML/CSS — Correct

Why is `box-sizing: border-box` often used globally?

Your answer: B. Declared width/height include padding and border, making sizing more predictable
Correct answer: B. Declared width/height include padding and border, making sizing more predictable
Why: With border-box, the specified width and height include content, padding, and border.

Q15 • JavaScript — Wrong

What does `typeof null` evaluate to in JavaScript?

Your answer: C. undefined
Correct answer: B. object
Why: typeof null returns the historical value "object". It is a well-known legacy quirk.

Q16 • JavaScript — Correct

Which statement about `const` is correct?

Your answer: B. A const variable cannot be reassigned after initialization, but an object it references may still be mutable
Correct answer: B. A const variable cannot be reassigned after initialization, but an object it references may still be mutable
Why: const prevents reassignment of the binding. It does not freeze referenced objects; object properties can still be mutated.

Q17 • JavaScript — Correct

What is the output? `console.log([1,2,3].map(x => x * 2).filter(x => x > 3));`

Your answer: B. [4,6]
Correct answer: B. [4,6]
Why: map produces [2,4,6]; filter keeps values greater than 3, giving [4,6].

Q18 • JavaScript — Correct

Which statement best describes a Promise's state after it is fulfilled?

Your answer: C. It is settled and cannot change state again
Correct answer: C. It is settled and cannot change state again
Why: Pending promises can settle as fulfilled or rejected. Once settled, the state is immutable.

Q19 • JavaScript — Correct

In an async function, what does `await promise` do?

Your answer: B. Suspends that async function's continuation until the promise settles, without blocking the event loop
Correct answer: B. Suspends that async function's continuation until the promise settles, without blocking the event loop
Why: await pauses the async function's execution flow while allowing the event loop to continue processing other work.

Q20 • JavaScript — Correct

What is event delegation primarily based on?

Your answer: A. Attaching one handler to a parent and using event bubbling to handle child events
Correct answer: A. Attaching one handler to a parent and using event bubbling to handle child events
Why: Event delegation places a handler on an ancestor and inspects the event target as events bubble upward.

Q21 • JavaScript — Correct

Which comparison avoids most implicit type coercion?

Your answer: C. ===
Correct answer: C. ===
Why: === compares type and value without the coercive conversions performed by ==.

Q22 • Angular — Correct

In Angular, which decorator marks a class as a component?

Your answer: B. @Component
Correct answer: B. @Component
Why: @Component defines metadata for a component, including its selector, template, and styles.

Q23 • Angular — Wrong

Which Angular mechanism is the standard choice for passing data from a parent component to a child component?

Your answer: C. Router
Correct answer: B. @Input()
Why: @Input properties allow data to flow from a parent component into a child component.

Q24 • Angular — Correct

What is `ngOnInit()` primarily intended for?

Your answer: A. Running logic after Angular initializes the component's input properties
Correct answer: A. Running logic after Angular initializes the component's input properties
Why: ngOnInit is a lifecycle hook called once after Angular has initialized data-bound input properties.

Q25 • Angular — Correct

Which RxJS operator is commonly used to cancel a previous HTTP search request when a new search term arrives?

Your answer: B. switchMap
Correct answer: B. switchMap
Why: switchMap unsubscribes from the previous inner observable when a new source value arrives, a common pattern for type-ahead HTTP calls.

Q26 • Angular — Correct

Which statement about Angular services is most accurate?

Your answer: B. Services are commonly used for shared business logic, state, or data access and are typically injected through DI
Correct answer: B. Services are commonly used for shared business logic, state, or data access and are typically injected through DI
Why: Angular services are injectable classes used to separate shared logic and data access from presentation code.

Q27 • Angular — Correct

What is the purpose of Angular route guards such as `CanActivate`?

Your answer: B. Control whether navigation to a route is allowed
Correct answer: B. Control whether navigation to a route is allowed
Why: Route guards can allow or prevent navigation based on authentication, authorization, or other conditions.

Q28 • Angular — Correct

In a reactive form, where should validation rules such as required and minimum length usually be defined?

Your answer: B. In the FormControl/FormGroup configuration using Angular validators
Correct answer: B. In the FormControl/FormGroup configuration using Angular validators

Why: Reactive forms define validation in the form model using validators, enabling programmatic and testable form rules.

Q29 • Angular — Wrong

Which strategy can reduce unnecessary view checks in a performance-sensitive Angular component?

Your answer: D. Replacing Observables with setInterval
Correct answer: A. ChangeDetectionStrategy.OnPush
Why: OnPush can limit change detection to relevant input/reference changes and explicit triggers, reducing unnecessary work.

Q30 • React — Correct

Why does React require a stable `key` when rendering a list?

Your answer: B. To help React identify which list items changed, were added, or removed
Correct answer: B. To help React identify which list items changed, were added, or removed
Why: Keys provide stable identity for list elements, helping React reconcile changes efficiently.

Q31 • React — Correct

Which hook is designed for storing component state?

Your answer: A. useState
Correct answer: A. useState
Why: useState adds state variables and a state update function to a function component.

Q32 • React — Correct

What is a common purpose of `useEffect`?

Your answer: A. Perform side effects such as subscriptions or data fetching after render
Correct answer: A. Perform side effects such as subscriptions or data fetching after render
Why: useEffect is used to synchronize a component with external systems, such as subscriptions, network calls, or DOM APIs.

Q33 • React — Correct

A child component receives a function prop created inline by the parent on every render. Which hook can help keep the function reference stable?

Your answer: B. useCallback
Correct answer: B. useCallback
Why: useCallback memoizes a function reference based on dependencies, which can help when child memoization depends on referential equality.

Q34 • React — Correct

Which statement about React state updates is correct?

Your answer: B. React may batch state updates, so updates should be expressed using functional updaters when they depend on previous state
Correct answer: B. React may batch state updates, so updates should be expressed using functional updaters when they depend on previous state
Why: React can batch updates. When next state depends on previous state, the functional updater form avoids stale reads.

Q35 • React — Correct

What does lifting state up usually mean in React?

Your answer: A. Moving the state to the nearest common ancestor so sibling components can share it
Correct answer: A. Moving the state to the nearest common ancestor so sibling components can share it
Why: Shared state is commonly moved to the nearest common ancestor and passed down via props or context.

Q36 • React — Correct

Which approach is generally preferred for large React applications that have many consumers of globally relevant data such as theme or authenticated user?

Your answer: B. Context or a dedicated state-management solution where appropriate
Correct answer: B. Context or a dedicated state-management solution where appropriate
Why: Context or state-management libraries can reduce excessive prop drilling when data is broadly shared.

Q37 • MongoDB — Correct

Which MongoDB operation is designed to update a single matching document?

Your answer: A. updateOne
Correct answer: A. updateOne
Why: updateOne updates at most one document matching the filter.

Q38 • MongoDB — Correct

What is the primary purpose of a MongoDB index?

Your answer: B. Reduce the amount of data the database must scan for supported query patterns
Correct answer: B. Reduce the amount of data the database must scan for supported query patterns
Why: Indexes provide data structures that can reduce document scanning for supported filters, sorts, and projections.

Q39 • MongoDB — Correct

Which MongoDB feature is most suitable for computing grouped totals across many documents?

Your answer: A. Aggregation pipeline with $group
Correct answer: A. Aggregation pipeline with $group

Why: The aggregation pipeline supports transformations and grouping, including $group for totals, counts, and other accumulations.

Q40 • MongoDB — Correct

Why can embedding related data be advantageous in MongoDB?

Your answer: B. It can allow related data to be read in one document and can avoid joins for common access patterns
Correct answer: B. It can allow related data to be read in one document and can avoid joins for common access patterns
Why: Embedding is useful when data is commonly accessed together and fits within document size and update-pattern constraints.

Q41 • MongoDB — Wrong

Which command is most useful for inspecting how MongoDB plans to execute a query?

Your answer: C. debugQuery()
Correct answer: A. explain()
Why: explain() provides query planning and execution statistics useful for performance analysis.

Q42 • MongoDB — Correct

In a MongoDB replica set, what is the typical role of a primary?

Your answer: B. It accepts the default write operations and replicates changes to secondaries
Correct answer: B. It accepts the default write operations and replicates changes to secondaries
Why: The primary is the default node for writes and replicates the oplog changes to secondary members.

Q43 • MongoDB — Correct

What does a unique index enforce?

Your answer: B. No two indexed documents can have the same indexed key value (subject to MongoDB's null/missing semantics)
Correct answer: B. No two indexed documents can have the same indexed key value (subject to MongoDB's null/missing semantics)
Why: A unique index enforces uniqueness of indexed keys, with special behavior for missing/null values depending on the index definition.

Q44 • Spring Core/AOP/Testing — Correct

What is the main benefit of dependency injection in Spring?

Your answer: B. It reduces direct coupling by supplying required dependencies from outside the class
Correct answer: B. It reduces direct coupling by supplying required dependencies from outside the class

Why: DI externalizes dependency creation and wiring, improving modularity, testability, and configurability.

Q45 • Spring Core/AOP/Testing — Wrong

Which Spring bean scope creates one instance per Spring application context by default?

Your answer: D. session
Correct answer: C. singleton
Why: singleton is the default bean scope in the Spring container, meaning one bean instance per container.

Q46 • Spring Core/AOP/Testing — Correct

In Spring AOP, a pointcut primarily defines what?

Your answer: B. Which method executions should be intercepted by advice
Correct answer: B. Which method executions should be intercepted by advice
Why: A pointcut expression selects join points, commonly method executions, where advice should run.

Q47 • Spring Core/AOP/Testing — Correct

Why might a Spring AOP advice not run when one method in a bean directly calls another method on `this`?

Your answer: B. Because proxy-based AOP may be bypassed by self-invocation
Correct answer: B. Because proxy-based AOP may be bypassed by self-invocation
Why: A direct self-invocation does not pass through the Spring proxy, so proxy-based advice may not be triggered.

Q48 • Spring Core/AOP/Testing — Correct

In a Spring Boot test, what is a common purpose of `@MockBean` (in setups that support it)?

Your answer: B. Replaces a bean in the application context with a Mockito mock
Correct answer: B. Replaces a bean in the application context with a Mockito mock
Why: @MockBean can replace or add a Mockito mock to the Spring application context, useful for isolating collaborators in tests.

Q49 • Spring DAO/Hibernate — Correct

What problem does Spring's declarative transaction management primarily solve?

Your answer: B. Coordinating transaction boundaries and commit/rollback without scattering transaction code everywhere
Correct answer: B. Coordinating transaction boundaries and commit/rollback without scattering transaction code everywhere

Why: Declarative transactions let developers specify transaction boundaries while Spring handles transaction lifecycle and rollback behavior.

Q50 • Spring DAO/Hibernate — Correct

What does lazy loading mean in Hibernate/JPA?

Your answer: A. Related data is loaded only when it is accessed, subject to the persistence context/session being available
Correct answer: A. Related data is loaded only when it is accessed, subject to the persistence context/session being available
Why: Lazy associations defer loading until access; accessing them after the persistence context is closed can cause problems such as LazyInitializationException.

Q51 • Spring DAO/Hibernate — Correct

Which JPA annotation normally marks the primary key field of an entity?

Your answer: A. @Id
Correct answer: A. @Id
Why: @Id identifies the entity's primary key property.

Q52 • Spring Boot/REST — Correct

Which annotation is commonly used on a Spring REST controller method to bind a JSON request body to a Java object?

Your answer: B. @RequestBody
Correct answer: B. @RequestBody
Why: @RequestBody tells Spring MVC to deserialize the HTTP request body into the method parameter.

Q53 • Spring Boot/REST — Correct

A REST endpoint creates a new resource successfully. Which HTTP status is most appropriate in the common case?

Your answer: B. 201 Created
Correct answer: B. 201 Created
Why: 201 Created is the conventional response when a request successfully creates a new resource.

Q54 • Spring Boot/REST — Correct

Which Spring Boot feature is most useful for exposing configuration such as database URLs via environment-specific properties without hard-coding them in Java?

Your answer: A. Externalized configuration using application properties/YAML and environment variables
Correct answer: A. Externalized configuration using application properties/YAML and environment variables

Why: Spring Boot supports externalized configuration through property/YAML files, environment variables, command-line arguments, and related mechanisms.

Q55 • Spring Boot/REST — Correct

What is a good reason to use `@ControllerAdvice` in a REST application?

Your answer: B. To centralize cross-controller exception handling and response mapping
Correct answer: B. To centralize cross-controller exception handling and response mapping
Why: @ControllerAdvice allows centralized exception handling and other controller-wide concerns, commonly with @ExceptionHandler.

Q56 • Microservices — Correct

A payment service calls an inventory service synchronously. Which issue is a key risk of this design?

Your answer: A. Network latency and cascading failures can propagate between services
Correct answer: A. Network latency and cascading failures can propagate between services
Why: Synchronous service-to-service calls introduce network latency, timeout, availability, and cascading-failure risks.

Q57 • Microservices — Correct

Which pattern is intended to prevent one failing downstream service from continuously consuming caller resources?

Your answer: A. Circuit breaker
Correct answer: A. Circuit breaker
Why: A circuit breaker can open after repeated failures and fail fast, protecting the caller and downstream system.

Q58 • Microservices — Correct

Why is centralized correlation or trace ID propagation important in microservices?

Your answer: B. It lets operators follow one business request across multiple services and logs/traces
Correct answer: B. It lets operators follow one business request across multiple services and logs/traces
Why: A correlation or trace ID links logs and traces across service boundaries, making distributed troubleshooting much easier.

Q59 • DevOps — Correct

In CI/CD, what is a major benefit of running automated tests on every change before deployment?

Your answer: B. It provides fast feedback and prevents known regressions from progressing down the pipeline
Correct answer: B. It provides fast feedback and prevents known regressions from progressing down the pipeline

Why: Automated tests provide rapid feedback and can block changes that violate expected behavior before they reach later environments.

Q60 • DevOps — Correct

In Docker, what is the practical difference between an image and a running container?

Your answer: B. An image is an immutable template/layer set; a container is a running or created instance of an image
Correct answer: B. An image is an immutable template/layer set; a container is a running or created instance of an image
Why: A container is created from an image and gets a writable runtime layer and process lifecycle, while the image is the reusable packaged template.

Restart assessment

Practice only: these are original IBM-style questions generated for your preparation, not leaked or official IBM assessment questions.